

2.10.43

Bhoomika L - EE25BTECH11014

August 2025

Question

Find the direction and normal vectors of the line $y = x + 2$

The equation of the line can be given by

$$\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix} \quad (1)$$

$$\mathbf{n}^T = \begin{pmatrix} 1 & -1 \end{pmatrix} \quad (2)$$

$$c = -2 \quad (3)$$

The equation of line can be expressed as

$$\mathbf{n}^T \cdot \mathbf{x} = c \quad (4)$$

where $\mathbf{n}$ is the normal vector of the given line.

Solution

The direction vector of the line is perpendicular to $\mathbf{n}^T$

$$\text{Let } \mathbf{m} = \begin{pmatrix} 1 \\ m \end{pmatrix} \quad (5)$$

We know that,

$$\mathbf{n}^T \cdot \mathbf{m} = 0 \quad (6)$$

$$\Rightarrow \begin{pmatrix} 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ m \end{pmatrix} = 0 \quad (7)$$

$$1 - m = 0 \quad (8)$$

$$m = 1 \quad (9)$$

$\therefore$ The normal vector of the line is $\mathbf{n}^T = \begin{pmatrix} 1 & -1 \end{pmatrix}$ and the direction vector

$$\mathbf{m} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

```
#include <stdio.h>

typedef struct {
    double x;
    double y;
} Vector2D;

// Function to compute direction and normal vectors
void computeVectors(Vector2D p1, Vector2D p2, Vector2D *direction
, Vector2D *normal) {
    direction->x = p2.x - p1.x;
    direction->y = p2.y - p1.y;

    normal->x = -(direction->y);
    normal->y = direction->x;
}
```

Python Code

```
import ctypes
import matplotlib.pyplot as plt
import numpy as np

# Define the Vector2D struct in Python to match the C struct
class Vector2D(ctypes.Structure):
    _fields_ = [(x, ctypes.c_double),
                (y, ctypes.c_double)]

lib = ctypes.CDLL('./code6.so')

# Set argument and return types for the function
lib.computeVectors.argtypes = (Vector2D, Vector2D, ctypes.POINTER(
    Vector2D), ctypes.POINTER(Vector2D))
lib.computeVectors.restype = None
```

```
# Create input points
p1 = Vector2D(1.0, 2.0)
p2 = Vector2D(4.0, 6.0)

# Prepare output vectors
direction = Vector2D()
normal = Vector2D()

# Call the C function
lib.computeVectors(p1, p2, ctypes.byref(direction), ctypes.byref(
    normal))

print(fDirection Vector: ({direction.x:.2f}, {direction.y:.2f}))
print(fNormal Vector: ({normal.x:.2f}, {normal.y:.2f}))
```

Python Code

```
# Line equation:  $y = x + 2$ 
# Normal vector = (1, -1)
# Direction vector = (1, 1)

# Define range for x
x_vals = np.linspace(-5, 5, 200)
y_vals = x_vals + 2 # original line

# Perpendicular line: slope = -1 (since slope of  $y=x+2$  is 1)
# Pass it through the y-intercept (0,2)
x_perp = np.linspace(-5, 5, 200)
y_perp = -x_perp + 2 # equation of perpendicular
```



```
# Plot
plt.figure(figsize=(6,6))
plt.plot(x_vals, y_vals, 'r-')
plt.plot(x_perp, y_perp, 'b-')

# Mark intercept
plt.scatter(0, 2, color='black', zorder=5)
plt.text(0.2, 2.1, '(0,2)', fontsize=9)

# Show direction and normal vectors at point (0,2)
plt.arrow(0, 2, 1, 1, head_width=0.2, head_length=0.3,
          fc='red', ec='red')
```

```
plt.arrow(0, 2, 1, -1, head_width=0.2, head_length=0.3,
          fc='blue', ec='blue')

# --- Label the original line with its equation directly on the
# line ---
plt.text(1, 1+2+0.2, r' $y = x + 2$ ', color='red', fontsize=11)

# --- Add text labels in the top-right corner ---
plt.text(3.5, 7, 'Red Original Line ( $y = x + 2$ )', color='red',
         fontsize=10)
plt.text(3.5, 6.3, 'Blue Perpendicular Line ( $y = -x + 2$ )', color='
blue', fontsize=10)
```

```
# Axes and grid
plt.axhline(0, color='gray', linewidth=0.5)
plt.axvline(0, color='gray', linewidth=0.5)
plt.gca().set_aspect('equal', adjustable='box')
plt.grid(True)

# Save as fig_6.png
plt.savefig('fig_6.png', dpi=300)
plt.show()
```

Plot

